# Regularising an LSTM Language Model on the Penn Treebank

*Andrea Giampietro*

University of Trento

## Abstract

We study how far a one-layer LSTM language model can be pushed on the Penn Treebank by adding one regularisation or optimisation technique at a time. Dropout and AdamW lower the test perplexity from 141.3 to 109.0; weight tying, variational dropout and non-monotonically triggered averaged SGD (NT-AvSGD) bring a model trained with plain SGD to 92.3.

## 1. Introduction

A language model estimates the probability of the next word given the previous ones, and its quality is measured by the perplexity (PPL), the exponential of the average per-token cross-entropy on held-out text. We train a word-level LSTM [1] on the Penn Treebank (PTB) [2, 3] and improve it incrementally, each step starting from the best model of the previous one. Part A adds dropout [4] and replaces SGD with AdamW [5]; part B keeps SGD and adds the three techniques of AWD-LSTM [6]: weight tying, variational dropout and NT-AvSGD. The code is available at `https://github.com/giampyhatesyou/Natural_Language_Processing`.

## 2. Implementation details

**Setup.** Every sentence is a sample ending with an `<eos>` token, the model predicts each next token and the padding is masked out of the loss. All the models use 256-dimensional embeddings, one LSTM layer with 256 units and a linear output layer over the 10k-word vocabulary. Gradients are clipped to norm 5, training stops when the dev PPL has not improved for 3 epochs, and the test set is always evaluated with the weights of the best dev epoch.

**Part A.** The two dropout layers ($p = 0.4$) act on the embeddings and on the LSTM outputs, before the output layer; the AdamW experiment uses the same model.

**Weight tying** [7, 8] reuses the embedding matrix $E$ as output layer, $\mathrm{softmax}(Eh_t + b)$: embedding and hidden sizes must match, and 2.56M of the 5.66M parameters disappear.

**Variational dropout** [9] samples one Bernoulli mask per sequence and applies it at every time step, on the same two positions as in part A, instead of a new mask for every token.

**NT-AvSGD** follows Algorithm 1 of [6] and wraps the SGD optimiser. Let $v_1, \dots, v_t$ be the dev perplexities of the previous epochs and $v$ the current one; averaging starts the first time that

$$t > n \quad \text{and} \quad v > \min_{i \le t-n} v_i, \tag{1}$$

with non-monotone interval $n = 5$: the model has to be worse than it was $n$ epochs before, so a single noisy epoch is not enough. From then on, after the $m$-th SGD step the weights $w_m$ update a running mean of the iterates [10],

$$\bar{w}_m = \bar{w}_{m-1} + \tfrac{1}{m}\,(w_m - \bar{w}_{m-1}), \tag{2}$$

and $\bar{w}$, not $w$, is validated, saved and tested. Early stopping is disabled before the trigger, which is itself a plateau detector, and applied to $\bar{w}$ afterwards with a patience of 5 epochs.

Table 1: *Part A: perplexity on the dev and test sets. All models have 5.66M parameters and are trained with batch size 64; Ep. is the epoch of the selected checkpoint.*

| Model | Optimiser | Ep. | Dev | Test |
|---|---|---|---|---|
| LSTM | SGD, lr 2 | 14 | 146.4 | 141.3 |
| + dropout (0.4) | SGD, lr 2 | 67 | 131.1 | 126.0 |
| + AdamW | AdamW, lr $10^{-3}$ | 42 | 121.2 | **109.0** |

Table 2: *Part B: perplexity on the dev and test sets. All models are trained with SGD (lr 1.5, batch size 32) and have 3.10M parameters; Ep. is the epoch of the selected checkpoint.*

| Model | Ep. | Dev | Test |
|---|---|---|---|
| LSTM + weight tying | 20 | 120.7 | 116.9 |
| + variational dropout (0.4) | 55 | 99.4 | 97.4 |
| + NT-AvSGD ($n = 5$) | 100 | 94.9 | **92.3** |

## 3. Results

Tables 1 and 2 report the dev and test PPL of one run per configuration (seed 42, trained on a T4 GPU), and Figure 1 the dev PPL after every epoch. The variance across seeds is not estimated, so small differences should be read with caution. In part A, dropout lowers the test PPL of the LSTM by 15.2 points and AdamW, on the same model, by a further 17.0, reaching 109.0. In part B the tied LSTM reaches 116.9 with 45% fewer parameters (the untied LSTM of part A, trained with a larger batch and learning rate, reaches 141.3), and variational dropout gives the largest gain of the study ($-19.5$). NT-AvSGD lowers the test PPL by a further 5.1 points, and the curves show where the gain comes from. Without early stopping before the trigger, SGD keeps improving after epoch 58, where the run without NT-AvSGD stopped, and its best iterate reaches 97.6 dev PPL at epoch 72; averaging, triggered at epoch 74, brings the dev PPL to 94.9 and is still improving at the 100-epoch limit. Of the 4.5 dev points gained over variational dropout, 1.8 come from the longer training and 2.7 from the averaging itself, in line with [6]. The final model, trained with plain SGD, is 16.7 test points better than the AdamW model of part A. As usually observed on PTB, the test PPL is lower than the dev PPL for every model.

## 4. References

[1] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

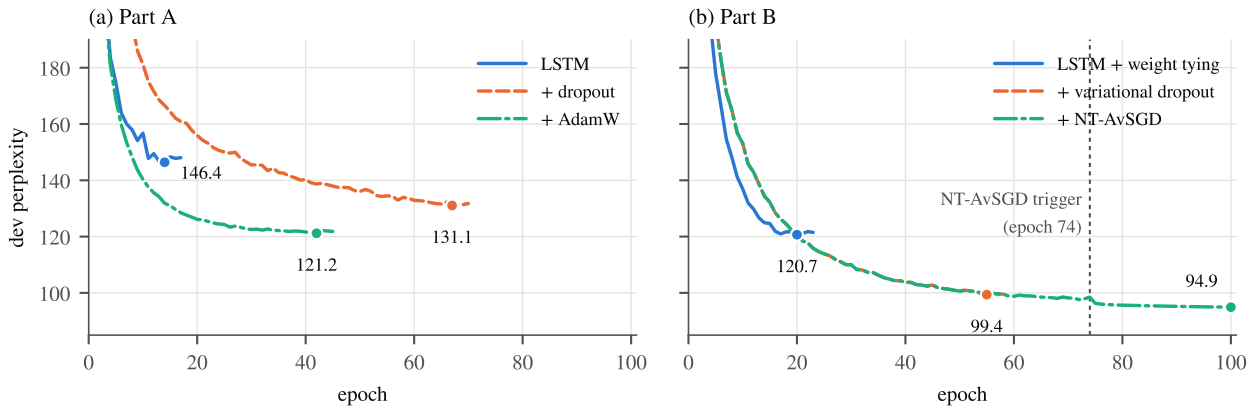



Figure 1: *Dev perplexity after every epoch (the first epochs, above 190, are cut). Dots mark the selected checkpoint of each run, i.e. its best dev epoch. The two runs with variational dropout coincide up to epoch 58, where early stopping ends the one without NT-AvSGD; with NT-AvSGD the averaged weights are evaluated from epoch 74 on.*


[2] M. P. Marcus, B. Santorini, and M. A. Marcinkiewicz, "Building a large annotated corpus of English: The Penn Treebank," *Computational Linguistics*, vol. 19, no. 2, pp. 313–330, 1993.

[3] T. Mikolov, M. Karafiát, L. Burget, J. Černocký, and S. Khudanpur, "Recurrent neural network based language model," in *Proc. Interspeech*, 2010, pp. 1045–1048.

[4] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.

[5] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. International Conference on Learning Representations (ICLR)*, 2019.

[6] S. Merity, N. S. Keskar, and R. Socher, "Regularizing and optimizing LSTM language models," in *Proc. International Conference on Learning Representations (ICLR)*, 2018.

[7] O. Press and L. Wolf, "Using the output embedding to improve language models," in *Proc. Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2017, pp. 157–163.

[8] H. Inan, K. Khosravi, and R. Socher, "Tying word vectors and word classifiers: A loss framework for language modeling," in *Proc. International Conference on Learning Representations (ICLR)*, 2017.

[9] Y. Gal and Z. Ghahramani, "A theoretically grounded application of dropout in recurrent neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.

[10] B. T. Polyak and A. B. Juditsky, "Acceleration of stochastic approximation by averaging," *SIAM Journal on Control and Optimization*, vol. 30, no. 4, pp. 838–855, 1992.